

- 1) Explain Cohesion and Coupling with their types.
- 2) Write notes on :- "User Interface Design". (Chp4)
- 3) Write notes on :- "Risk Management".
- 4) Explain Size Oriented Software engineering Metrics.
- 5) Explain Function Point Estimation Technique.
- 6) Explain Integration Technique.
- 7) Explain System Testing.
- 8) Differentiate betn Verification and Validation Testing. (Chp6)
- 9) Differentiate betn White box testing and Black box testing. (Chp6)
- 10) Write notes on :- "Reverse Engineering". (Chp6)

+

SE :- Assignment 2. Submit on: 11/9/25

- 1) Explain Cohesion & Coupling with their types.
- 2) Write notes on :- "User Interface Design".
- 3) Write notes on :- "Risk Management".
- 4) Explain different metrics used for Maintaining software quality.
- 5) Differentiate between Verification & Validation Testing.
- 6) Differentiate betn White box testing & Black box testing.
- 7) Write notes on :- "Reverse engineering".

Total :- 8/11

Verificn :- 2 kabhi wrong, 3 kabhi
Valid :- user are internal to the system.

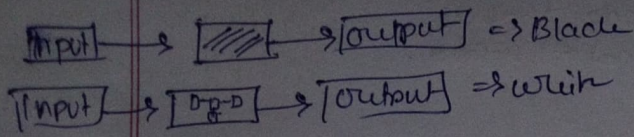
2 ka likha, 3 ka nanga

Date: valid (x) / (x) / (x)
Page: Ver (x) / (x) / (x)

* Answers :-

Q1 Differentiate betn Verification and Validation Testing.

Parameters	Verification	Validation
Focus	"Are we building the product right?"	"Are we building the right product?"
Defn	Verification ensure that the software product is designed & developed in a correct track of creating the end user product.	Validation can be defined as it is process of evaluating the final product to ensure that the software meets all the specified req. of users.
Testing	Verification is static Testing.	Validation is dynamic Testing.
Atm	Process Oriented	Product Oriented.
Time When hrs	Verification take place before Validation.	Validation after
Inclusion	Verification contains Reviews, meetings & Inspections.	Containing blackbox, white, graybox testing, etc.
Execution of code	Execution of code is not a part	Execution of code is part
Performer	Conducted by quality assurance team	Conducted by Testing team.



Q7 Differentiate betn: Whitebox Testing & Blackbox Testing

<u>Parameters</u>	<u>Whitebox Testing</u>	<u>Blackbox Testing</u>
<u>Aim</u>	It aims at how system is performing its functionality.	It aims at what functionality is system performing.
<u>Performed by</u>	Carried out by soft-developers.	Carried out by s/w testers.
<u>Cost</u>	More Expensive	Less Expensive
<u>Applicable levels</u>	At lower level i.e. Unit & Integration testing.	At Higher level i.e. Acceptance & System testing.
<u>Focus on</u>	Test code, security of s/w & flow of control.	do not test code, focus on functionality of s/w prod.
<u>Time</u>	Time consuming requires expertise.	Limited coverage may miss internal bugs.
<u>Knowledge</u>	Internal knowledge of code are known.	Not known.
<u>Aka.</u>	Clear box testing.	closed testing.

Design Dehke ke src code.
src code sketch ke Maya design

(Idea ko Guzari hai, or H 3-4 or)

Date _____
Page _____

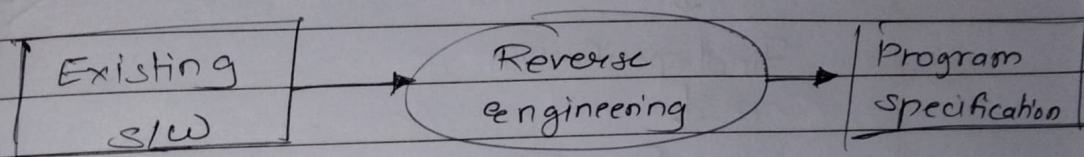
lv = level

Write note on : "Reverse Engineering"

Reverse Engineering is the procedure to get system specification by analyzing and understanding the existing system.

Consider an existing system has design about which we donot know anything. System designers performs reverse engineering by analyzing the source code and attempt to get design.

With the help of design, designer attempt to make conclusion about specifications of S/W product.



The Reverse Engineering has following Purposes:-

- 1) Used of learning tools.
- 2) Security auditing.
- 3) Enabling additional features.

Following are the imp. parameters / 3-Levels :-

1) Abstraction level :

- Highest level

- Design information is extracted from src. code.

2) Completeness :

- nothing but details available through Abstractⁿ lv.

3) Directionality :

- giving or making available the extracted info. from source to software developers.

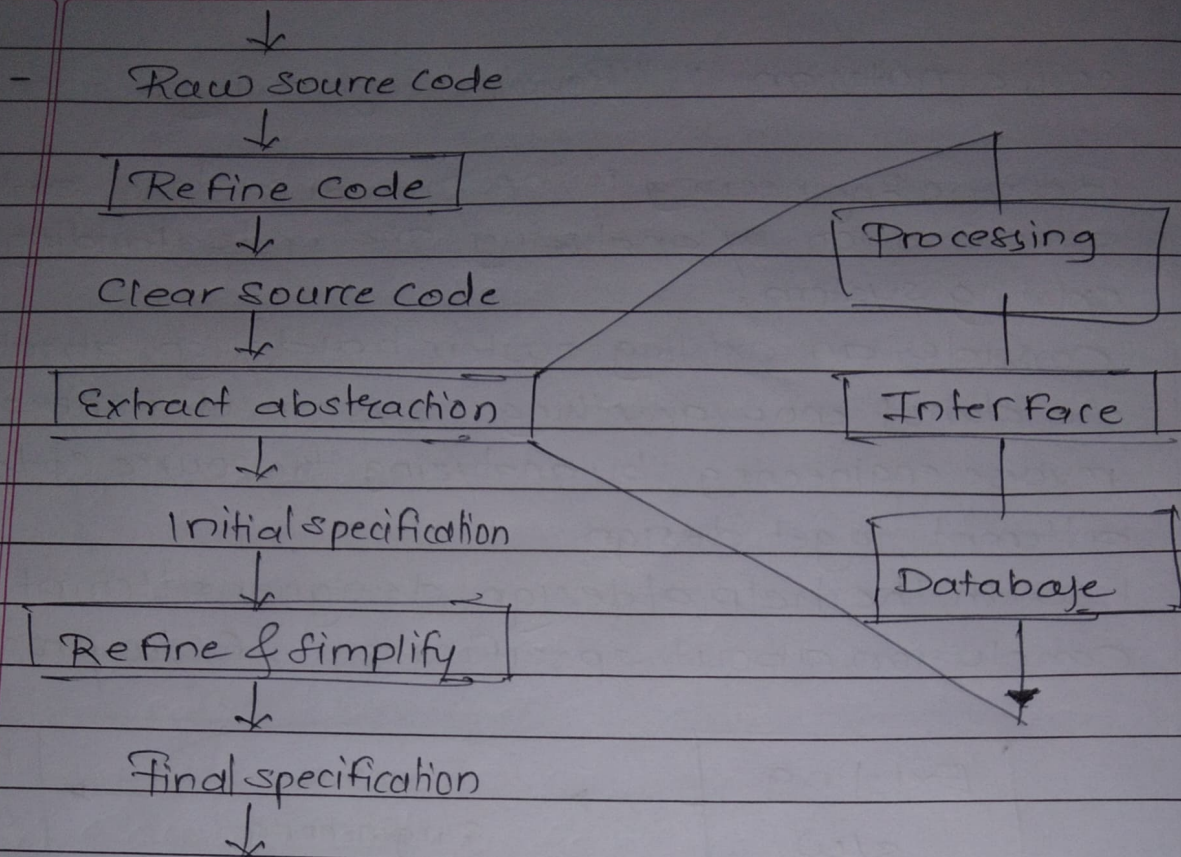


fig: Reverse Engineering Process.

Following are steps :-

- Raw/dirty source code obtained from the abstraction lvl is taken as Input.
- This code is Refined or Restructured.
- From Clean source code, abstraction is extracted.
- From this initial specification is refined and simplified.
- Now we get final specification.

Q. Explain cohesion & coupling with their types.
Both cohesion & coupling are s/w designed principles used to measure how well a module of a s/w system is design.

• Cohesion :- (गुणवत्ता, उसी dependencies को)
Function / elements / parts / component system

- Cohesion define the degree to which components of system are related to each other i.e. measure the strength of relationship betn two components of systems.
- Good system design must have High Cohesion betn the components of systems.
- Types of Cohesion :

i) Functional Cohesion	High
ii) Sequential Cohesion	
iii) Communicational Cohesion	CLTPCSF
iv) Procedural Cohesion	-----
v) Temporal Cohesion	
vi) ^{logical} Co-incident Cohesion	
vii) Co-incident Cohesion	low

i) Co-incident Cohesion :

- An unplanned Cohesion, parts of module are grouped arbitrarily (randomly)
- No logical relationship.
- Quality - worst. e.g., different Dept. students together.

ii) Logical Cohesion :

- All the elements of a module perform similar or slightly similar operations. (similar kind of act, diff. nature)
- Quality - poor
- e.g., keyboard, mouse, scanner.

iii) Temporal Cohesion:

- In case of Temporal Cohesion functions are related by a fact that all task must be executed in the same time span for flow control in betn.
- Quality $\rightarrow$ Low
- eg., two funcⁿ; 1 catches initiation of other termination activation.

iv) Procedural Cohesion:

- Here components of system are related to each other through sequence / flow control.
- Quality $\rightarrow$ Good.
- eg., two funcⁿ; 1 checks the file access permission & 2nd opens the file.

v) Communication Cohesion:

- when elements of module perform different funcⁿs but each funcⁿs accepts same I/P and generates O/P.
- Quality $\rightarrow$ Good
- eg., Push and pop.

vi) Sequential Cohesion:

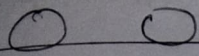
- A module is said to be sequentially cohesive if its functions are related in a way such that output of one funcⁿ acts as an input data to other funcⁿ.
- Quality $\rightarrow$ better
- eg., 1 funcⁿ provides data, 1 process data.

vii) Functional Cohesion:

- Elements of Module are grouped together, since they contribute to a single function.
- Quality $\rightarrow$ best.
- all elements are necessary
- eg., mouse has diff funcⁿ:- track locⁿ, appropriate cursor movement.

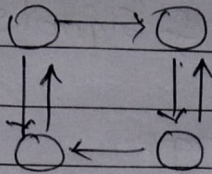
Coupling :-

- measure of interdependence of one module over another module.
- Good design always have low coupling betn components of systems.



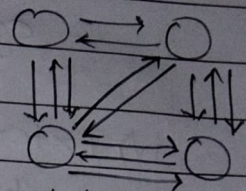
Uncoupled:

No dependencies



Low Coupled:

Less dependent



Highly coupled:
more dependent.

- Types of Coupling :

- Data Coupling
- Stamp coupling
- Control coupling
- External coupling
- Common Coupling
- Content Coupling

Best

Worst

C3DSE

CCECSD

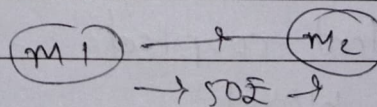
DSC ECC

i) Data Coupling :

- Coupling between 2 modules when they pass data to each other by parameter using argument

- Quality → Best

- eg.:



'Independent' ऑन लाईन ऑन-कॉल.
call by value

ii) Stamp Coupling : RTA

- passing an entire record, but only using one field.

- Quality → Good.

→ eg. call by ref.

iii) Control coupling:

- One module can control the internal logic of another by passing control information.
- passing flag can change how another module behaves.
- Quality $\rightarrow$ Moderate
- eg., train & signal system $\rightarrow$ $\frac{1}{2}$ at 3rd level, but still depend

iv) External coupling:

- two components of a system share an externally imposed data format, communication protocol or device interface
- Quality $\rightarrow$ Poor.
- eg., two train with Green signal on single track

v) Common Coupling: / global coupling

- Two modules are set to be common coupled if they share some global data.
- Quality $\rightarrow$ Very poor.
- eg., process synchronization

vi) Content coupling:

- When one component refers to the internal of other component then these two components are said to be content coupled.
- Quality $\rightarrow$ Worst
- eg., car and its engine.

Q. Write notes on: "User Interface Design".

- User interface is considered as a frontend applⁿ view which is interacted by the user so as to use a software.
- User is able to manipulate as well as control the software and hardware with the help of User Interface.
- It focuses on how information is presented and how user interact with system, ensuring that the experience is both efficient and satisfying.
- Goals of UI Design: -
 - 1) Attractive
 - 2) Simplicity - Avoid unnecessary complexity
 - 3) Responsive in short time
 - 4) Clarity - Interface design should clearly present funcⁿ, instructⁿ, feedback.
 - 5) Consistency - Similar elements should behave in predictable ways.
 - 6) Error prevention - Minimize chance of error
 - 7) Feedback - System should respond to user action.
- Types of Interface :-
 - 1) Command line Interface (CLI)
 - 2) Menu-Based Interface
 - 3) Graphical User Interface (GUI)
 - 4) Touch/Voice Interface
- UI Design Process :-
 - 1) Identify user needs
 - 2) Analyze tasks & user behaviour.
 - 3) Design Layout and Interaction

- 4) Build prototype.
- 5) Evaluate & Improve based on feedback.

- UID Analyze to design :-

① User Analysis :

- understands who the users are (age, skill, expectns)

② Task Analysis :

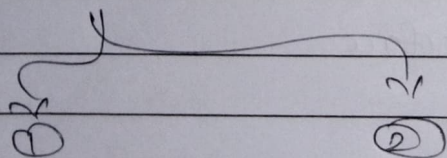
- Study the task user perform
- Complexity, steps, Sequence

③ Environment Analysis :

- Where and how users will interact
- mobile, desktop.

87 → Write note on: "Risk Management".

- A Risk is an uncertain event or condition that, if it occurs, has a positive or negative effect on a project's objectives.
- Risk management in S/W engineering is the process of identifying, analyzing & mitigating risks that can negatively impact a project.
- Strategies



Reactive

Proactive.

① Reactive :- tries to reduce the damage of potential threats & speed an organizⁿ recovery from them but assume that those threat will happen eventually.

② Proactive: identifies threats and events to prevent those events from ever happening in first place.

- Types of Risks :-

i) Business Risk:- Building a product that no one wants or losing budgetary commitments.

ii) Technical Risks: Concerned with quality, design, complexity, maintenance problems.

iii) Project Risk: Concerned with schedule, cost, resources, staffing issue.

iv) Operational Risk: Environment or H/W Failure.

- Steps :-

i) Risk Identification: find possible risks.

ii) Risk Analysis: Assess probability of impact.

iii) Risk prioritization: Rank risks by severity.

iv) Risk Mitigation: Develop plans to avoid risk.

v) Risk monitoring: Continuous track risks throughout project.

Q7

Explain Size Oriented Software Engineering Metrics.
Size - Oriented Metrics S/W development based on size of S/W typically in Line of Code (LOC)

Common Size Metrics.

• LOC - Count total executable lines

• KLOC - Thousand of lines of code.

• Program Size - Total st. in code

• Example

1. productivity = $\text{LOC} / \text{Person-month}$
2. quality = $\text{Defect} / \text{KLOC}$
3. cost = $\text{cost} / \text{KLOC}$
4. Documentⁿ = $\text{pages} / \text{KLOC}$

- Adv:

- simple to measure & understand,
- Easy to compare productivity b/w project

- Disadv/Limitatn:

- Not suitable for non-coding phase (design) testing.
- Encourage writing longer code unnecessarily.

Explain different metrics used for maintaining Software quality.

Software quality metrics are standard of measurement that help evaluate the quality of software in terms of performance, reliability, usability and maintainability. These metrics guide developers and testers to ensure that the software meets customers' expectation and industry standard.

1) Product Quality Metrics :

These measure the quality of the software product itself.

- a) Correctness - Measure how well the software performs its intended function.
- b) Reliability - Probability that software will operate without failure over a specific too time.
- c) Usability - Ease with which users can learn and operate the software.
- d) Efficiency - Measure how well the system uses resources like CPU, time, memory, and response, time.

2) Process Quality Metrics :

These focus on the processes used in software development & maintenance.

a) Defect Removal Efficiency (DRE) -

$$\left(\frac{\text{Defects Removal before Release}}{\text{Total defects}} \times 100 \right)$$

b) Review Efficiency - Effectiveness of code reviews and inspection.

c) Process Productivity - Output (LOC or function points) per unit effort (person-hours)

3) Project Quality Metrics:

These measures how well the project is managed & controlled

a) Effort Variance =

$$\frac{(\text{Actual effort} - \text{Estimated effort})}{\text{Estimated effort}} \times 100$$

b) Scheduled Variance = $\frac{(\text{Actual time} - \text{Planned time})}{\text{Planned time}} \times 100$

c) Cost Variance = $\frac{\text{Actual cost} - \text{Budgeted cost}}{\text{Budgeted cost}} \times 100$

d) Defect Density - Number of defects per KLOC (thousands time lines of codes) or per function time.

4) Maintenance Quality Metrics:

These deal with software after deployment

a) Maintainability - Ease of correcting defects or making improvement.

b) Mean Time to Repair (MTTR) - Average time, to fix a defect.

- c) Change Request Frequency - How often change are requested after release.
- d) Code claim - Amount of code modified over time
(too much claim may mean instability)

⑦ Explain System Testing.

→ System testing is the process of testing the entire integrated system to verify it meets the specified requirements. It is black box testing.

Types of System testing.

1. Functional Testing: Verifies functional requirement ✓

2. Performance testing: Tests speed, Stability, Scalability.

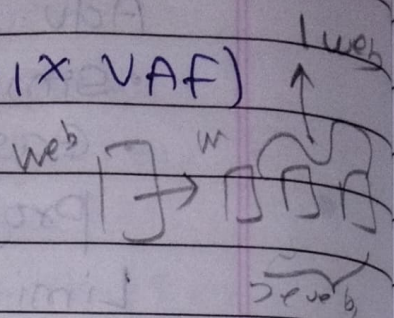
3. Security Testing: Ensure system is protected from threats. - virus - web

4. Recovery testing: checks system recovery from crashes

5. Usability Testing: Evaluate user - friendliness, User related objective.

- Validate that the SIW works as a whole
- Ensure it meets business & technical requirements

3. Compute Unadjusted Factor (UFP) = $\sum (\text{Count} \times \text{Weight})$
4. Calculate Value Adjustment Factor (VAF):
Based on 4 general system characteristics.
5. Final FP values
$$FP = UFP \times (0.5 + 0.01 \times VAF)$$



⑥ Explain Integration Technique.

white box testing used

→ Integration testing is the process of combining individual SW modules & testing them as a grp to ensure they work together correctly. It occurs after Unit & before System testing.

1. Big Bang Integration.

- All modules are combined and tested together at once.

- Easy but hard to isolate errors.

2. Incremental Integration.

- modules are integrated step by step.

- Easier to identify defects.

Types of Incremental Integration.

- Top-Down Testing start from top-level module; stubs used for lower ones.

- Bottom up: Testing starts from lower module; drivers used for upper ones.

- Sandwich: combine both top-down & bottom-up approach.

Adv. Disadv.